

CHESTNUT: 이동형 MEC QoS 데이터셋 읽기

CHESTNUT: A QoS Dataset for Mobile Edge Environments

Guobing Zou · Fei Zhao · Shengxiang Hu (2024)

Shanghai 위치·이동 trace를 바탕으로 MEC workload와 QoS를 생성한 arXiv v3 데이터셋 논문

모바일 MEC QoS에는 시간과 위치가 함께 필요하다

같은 서비스라도 사용자 이동, 서버 coverage, 부하가 바뀌면 QoS 문맥이 달라진다.

01 정적 matrix user·service QoS만 남기기 쉬움



02 이동 사용자 위치와 요청 시각이 변함



03 에지 서버 coverage와 부하가 변함



04 CHESTNUT 이 문맥을 한 dataset에 연결

목표는 MEC QoS 예측용 문맥을 제공하는 것이지 특정 예측기의 승리를 보이는 것이 아니다.

먼저 구분할 것: trace seed와 생성된 QoS

CHESTNUT은 두 Shanghai source dataset을 그대로 합친 raw telemetry table이 아니다.

구성	논문이 사용하는 입력	CHESTNUT에서의 상태
사용자 이동	Taxi GPS 위치·속도·방향	정렬·선별한 user trace
후보 서버	Telecom base-station 좌표	radius·supply를 부여한 server
서비스·호출	원자료 없음	demand·request를 합성
load·rt·nj	원자료 없음	생성 모델의 QoS 출력

생성 요소를 실측 operator QoS로 부르면 데이터셋의 근거 범위를 넘는다.

두 Shanghai source trace의 역할

source의 규모와 사용 범위를 함께 읽어야 proxy를 실제 MEC 측정으로 오해하지 않는다.

source	논문이 말한 범위	CHESTNUT에서의 역할
Johnson Taxi	특정 하루 GPS와 이동 상태	mobile user trace seed
Shanghai Telecom	7.2M records, 9,481 phones, 3,233 sites	base-station location seed
Telecom 사용 범위	one month·Shanghai geography	candidate server simulation
공개 release	paper가 Kaggle link를 명시	dataset access path

택시와 기지국 좌표는 mobility·location proxy이며, 실측 MEC capacity trace가 아니다.

CHESTNUT 구성 pipeline

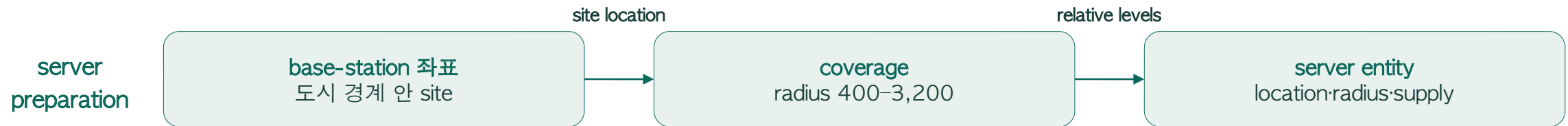
trace의 provenance와 generator의 output을 한 흐름에 보되, payload를 분명히 적는다.



마지막 상자의 'rt,nj'는 trace에서 직접 읽은 열이 아니라 generator가 만든 QoS output이다.

후보 edge server 만들기

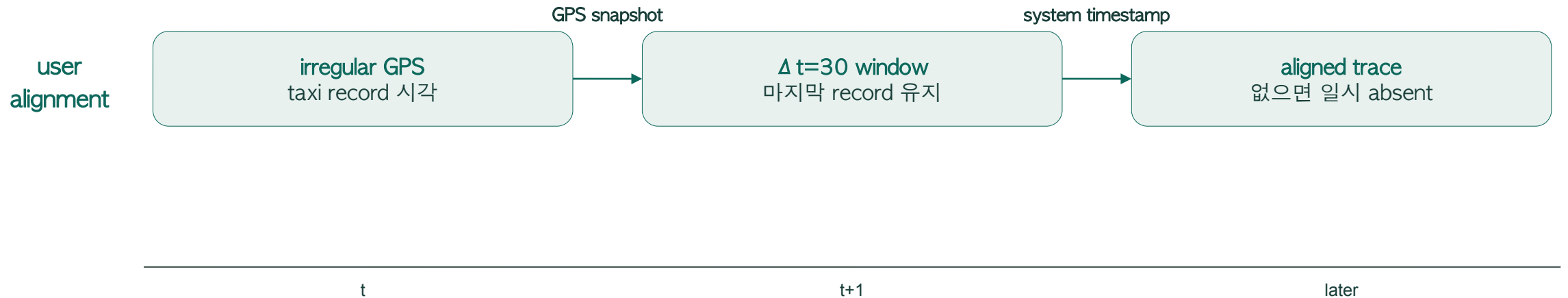
기지국 좌표에 coverage와 상대 supply를 부여해 MEC server entity를 구성한다.



radius와 compute·storage·bandwidth level은 논문의 simulation configuration이다.

불규칙 GPS를 고정 시점 user trace로 정렬

이 time axis는 논문이 명시한 fixed-window snapshotting 절차를 보여 준다.



최종 user trace는 raw taxi table이 아니라 alignment와 screening 뒤의 연구용 표본이다.

합성 서비스의 수요 수준

각 `sid`는 compute, storage, bandwidth의 세 상대 demand level을 갖는다.

field	값의 성격	논문 규칙
sid	synthetic service ID	총 1,000 service
computing	relative demand	1..Ps, Ps=4
storage	relative demand	1..Ps, Ps=4
bandwidth	relative demand	1..Ps, Ps=4
같은 tuple	서로 다른 sid 가능	latent app 차이를 모사

같은 demand tuple도 다른 service ID가 될 수 있다.

QoS 예측의 단위: user-server-service-time

paper는 invocation을 $R_i = \langle u, e, s, t \rangle$ 로 정의하고 sample rows에 QoS output을 붙인다.

entity	key/state	예시 field
user	u at timestamp t	lon, lat, speed, direction
server	e covering u	location, radius, supply, load
service	s at t	compute, storage, bandwidth
invocation	(u,e,s,t)	uid, eid, sid, timestamp
generated QoS	invocation output	rt, nj

random row split은 같은 user-server의 미래 state나 관계를 test에 섞을 수 있다.

요청 시퀀스와 invocation 생성

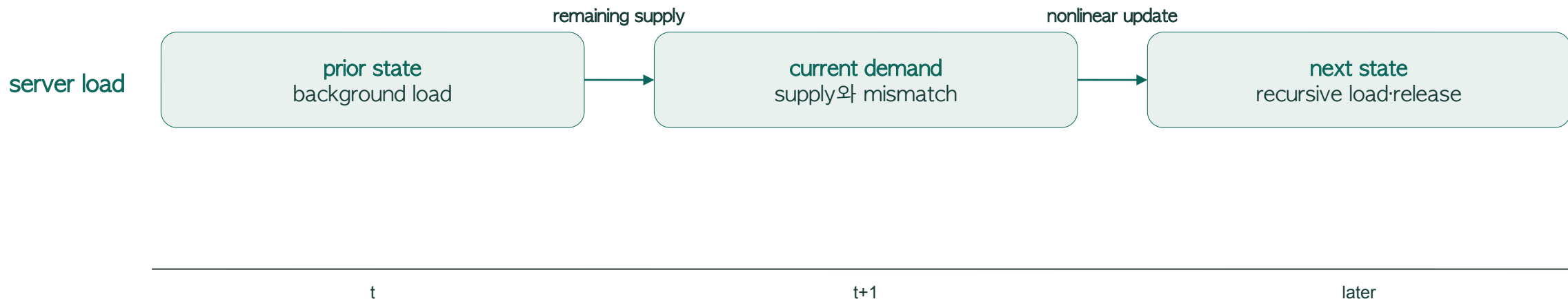
공통 리듬과 개인 variation을 만들고, coverage 조건에서 server와 service를 연결한다.



같은 group도 user별 request trace는 다르다.

server load는 매 시점 독립 표본이 아니라 재귀 state다

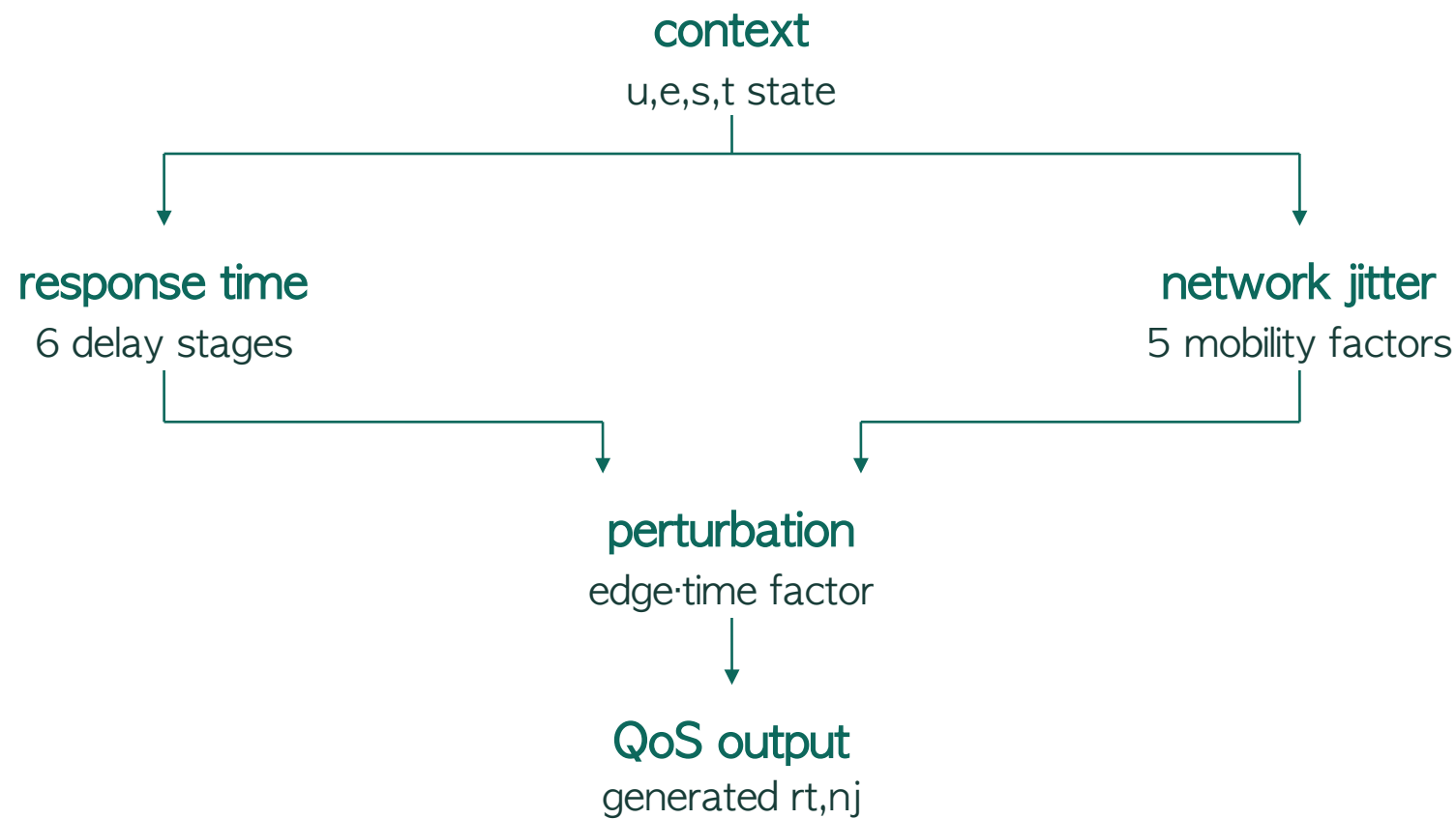
이 time axis는 Section III-B의 prior load, current demand, release update를 압축한다.



load history는 forecast에 유용하지만 random split 누수를 부른다.

response time·jitter 생성

둘 다 user-server-service-time 문맥을 쓰지만, paper의 생성 기제는 서로 다르다.



가상 예시: coverage에서 QoS context key까지

아래 값은 CHESTNUT 원본 행이 아닌, schema를 손으로 읽기 위한 invented example이다.

항목	가상 값	해석
context key	uid=7, eid=2, sid=10, t=4	한 invocation 문맥
coverage	distance 0.6 km, radius 1.0 km	$0.6 \leq 1.0$ 이므로 cover
service demand	(compute, storage, bandwidth)=(3,2,4)	relative levels
forecast target	generated rt,nj	과거 state만 input으로 사용

가상 key는 원본 QoS event를 재현하지 않는다.

Table VII: CHESTNUT의 전체 규모

이 숫자는 dataset inventory이며, model accuracy나 deployment capacity가 아니다.

entity	count	읽는 방법
users	1,000	alignment-selection 뒤 user trace
servers	1,763	candidate edge-server entities
services	1,000	synthetic service IDs
timestamps	720	aligned system index
invocations	33,551,574	user-server-service-time rows

큰 row 수가 real operational telemetry의 외적 타당성을 자동으로 보장하지는 않는다.

mobility와 coverage가 만드는 시간 의존성

Figures 6–7은 selected trace와 request-pattern generator의 성질을 보인다.

관찰	paper의 수치/설명	평가상 의미
평균 coverage 1	1,638 servers	희소하지만 overlap 존재
평균 coverage 2	107 servers	coverage 관계를 feature로 사용
평균 coverage 3 / 4+	13 / 5 servers	분포의 tail도 유지
same request group	major rises/falls는 유사	user trace는 동일하지 않음

같은 user-server-group의 상관을 무시한 random split은 미래 정보를 섞을 수 있다.

생성 결과의 분포: QoS model accuracy와 구분

Figures 8–1 1은 generator가 만든 service/load/response/jitter의 descriptive statistics다.

항목	reported statistic	해석 범위
service levels	각 resource level 약 250 services	synthetic demand 균형
mean load	compute 78.6%, storage 60.5%, bandwidth 52.8%	representative server
response time	31.2% <20 ms; 40.0% at 20–40 ms	mean 60.9, median 25.8 ms
network jitter	51.7% <40 ms; 34.6% at 40–80 ms	mean 48.7, median 39.0 ms

분포 통계는 predictor generalization 검증과 다르다.

WS-DREAM과의 비교는 feature availability 표다

Table VIII의 체크는 데이터 항목의 존재를 말하며 benchmark quality ranking은 아니다.

property	WS-DREAM	CHESTNUT
user-server-service QoS records	—	제공
user trajectory-coverage	—	제공
supply-load-demand-jitter	—	제공
response time	제공	제공
throughput	제공	—

차별점은 MEC 문맥 field이며, 표가 예측 성능을 증명하지는 않는다.

schema에서 만들 수 있는 조건부 연구 과제

paper가 언급한 MEC uses를 dataset의 실제 key와 context에 연결한다.

01 QoS forecasting 과거 context로 rt, nj 예측



02 service selection coverage 후보 안에서 선택



03 load-aware simulation server state 변화 분석



04 평가 설계 time·entity split을 선언

CHESTNUT은 context-rich evaluation substrate로 쓸 수 있다.

평가 설계: row split 전에 막아야 할 누수

아래는 paper의 prescribed protocol이 아니라 schema를 읽어 도출한 evaluation safeguards다.

의존성	naive split의 위험	명시할 분리
timestamp	미래 load·요청 추세 노출	chronological cutoff
same user	trajectory history 재등장	user-held-out 또는 history rule
same server	coverage·load relation 재등장	server/region generalization
request group	shared trend가 test에 섞임	group information 공개 여부 확인

무엇을 generalize하려는지 먼저 정해야 split도 해석할 수 있다.

사용 범위와 한계

dataset의 강점인 controllable MEC context가 동시에 generalization boundary를 만든다.

근거가 있는 주장

Shanghai trace-seeded mobility/location context

model-generated response time와 jitter

sampled radius·relative supply

Table VII inventory와 Fig. 8-11 분포

feature comparison with WS-DREAM

근거가 없는 확장

다른 도시·carrier·user population의 대표성

operator telemetry와 동등한 실측 QoS

실제 MEC hardware/radio capacity

energy, security, packet-loss benchmark

predictor accuracy ranking

실제 network telemetry로의 validation을 별도 수행해야 deployment claim을 할 수 있다.

CHESTNUT을 재현 가능하게 읽는 순서

source boundary, entity key, generator, evaluation scope를 순서대로 확인한다.

01 **provenance** trace seed와 생성 output 분리



02 **schema** u,e,s,t와 rt,nj 확인



03 **generator** load·response·jitter 기제 확인



04 **evaluation** split·city·telemetry 경계 명시

CHESTNUT의 가치는 dynamic MEC context와 생성 가정을 함께 보이는 데 있다.